

Arquitectura Transformer en profundidad

Módulo 1 · Fundamentos Ontológicos y Razonamiento de Máquinas

Del paper "Attention Is All You Need" al GPT-4: embeddings, self-attention, bloques Transformer, encoder vs decoder y coste cuadrático del contexto.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Arquitectura Transformer en profundidad

En junio de 2017, ocho investigadores de Google publicaron un paper de 15 páginas con el título "Attention Is All You Need" (Vaswani et al., arXiv:1706.03762). Propusieron una nueva arquitectura de red neuronal —el Transformer— basada exclusivamente en mecanismos de atención, sin recurrencia ni convoluciones. En los benchmarks de traducción automática, el modelo superó todos los resultados anteriores mientras era significativamente más paralelizable y requería menos tiempo de entrenamiento.

Ese paper es posiblemente el más influyente de la historia reciente de la IA. Todos los LLMs modernos —GPT, Claude, Gemini, Llama, Mistral— son variantes de la arquitectura Transformer. Entender el Transformer no es solo de interés académico: es la base para entender por qué los LLMs tienen las capacidades que tienen (comprensión de contexto largo, relaciones a larga distancia en el texto) y las limitaciones que tienen (coste cuadrático del contexto, "lost in the middle").

Los componentes del Transformer

Embeddings e input encoding: de texto a vectores

El primer paso en el procesamiento de un Transformer es convertir los tokens (unidades discretas de texto) en vectores continuos (embeddings). Cada token del vocabulario tiene un embedding asociado, que se aprende durante el entrenamiento. Además, se añade un positional encoding: una señal que indica la posición de cada token en la secuencia. Sin él, el Transformer trataría "El perro mordió al niño" y "El niño mordió al perro" como secuencias idénticas (porque los mecanismos de atención son invariantes al orden sin esta señal).

El Transformer Block: la unidad repetible

La arquitectura Transformer se construye apilando N bloques idénticos (entre 12 y 96 o más en los modelos modernos). Cada bloque tiene dos componentes principales: un módulo de multi-head self-attention y un módulo de feed-forward network (FFN). Cada componente está rodeado de una conexión residual (que suma la entrada al output del componente) y normalización de capa. La conexión residual es fundamental: permite al gradiente fluir directamente desde las capas más profundas a las más superficiales durante el backpropagation, haciendo posible entrenar redes de decenas o cientos de bloques.

Feed-Forward Network y proyección de salida

El FFN en cada bloque del Transformer aplica dos transformaciones lineales con una no-linealidad (GELU o ReLU) en el medio: $\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 \cdot x + b_1) + b_2$. Aunque aparentemente simple, el FFN es donde el modelo almacena gran parte del "conocimiento factual" aprendido durante el preentrenamiento, según investigaciones recientes sobre interpretabilidad. La dimensión de la capa oculta del FFN es típicamente 4 veces la dimensión del modelo.

El detalle técnico del procesamiento completo

El flujo completo de procesamiento en un LLM decoder-only (como GPT) es: 1) tokenización del texto de entrada en IDs de tokens; 2) conversión de IDs a embeddings + positional encoding; 3) paso por N bloques del Transformer, cada uno con self-attention y FFN con conexiones residuales; 4) normalización de capa final; 5) proyección a logits sobre el vocabulario (un vector de tamaño V donde V es el tamaño del vocabulario); 6) softmax para convertir logits en probabilidades sobre el siguiente token; 7) muestreo del siguiente token según las probabilidades (con temperatura para controlar la aleatoriedad).

En cada bloque, la self-attention calcula para cada token una representación contextualizada que pondera la información de todos los demás tokens de la secuencia según su relevancia. Esto es lo que permite al modelo entender que "banco" en "me senté en el banco del parque" significa algo diferente que en "abrí una cuenta en el banco", dependiendo del contexto.

Encoder vs Decoder: dos variantes del Transformer

El Transformer original tenía encoder y decoder. El encoder procesa toda la secuencia bidireccional (cada token puede atender a todos los demás, tanto anteriores como posteriores). BERT es un encoder-only: se usa para tareas de comprensión (clasificación, extracción, embeddings). Los modelos GPT son decoder-only: usan masked attention donde cada token solo puede atender a los tokens anteriores (causales), lo que permite la generación autoregresiva de texto. Los modelos T5, BART son encoder-decoder: el encoder procesa la entrada, el decoder genera la salida, útiles para traducción y resumen.

SECCIÓN 4 · EJEMPLOS REALES

- GPT-4: decoder-only con ~1.8T parámetros (estimación), ~128K tokens de contexto, multi-head attention con ~128 cabezas. La escala es lo que produce las capacidades emergentes de razonamiento.
- BERT base: encoder-only, 12 bloques, 768 dimensiones de embedding, 12 cabezas de atención, 110M parámetros. Usado para embeddings, clasificación de texto, NER.
- Llama 3 8B: decoder-only con arquitectura moderna (grouped query attention, RoPE positional encoding, SwiGLU activación). Open weights, deployable en hardware de consumo con quantización.
- Vision Transformer (ViT): aplica la arquitectura Transformer a imágenes dividiendo la imagen en patches (tokens visuales). Usado en modelos multimodales como GPT-4V, Claude Sonnet, Gemini.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: "El Transformer entiende el texto como un humano." No. Procesa secuencias de tokens y produce representaciones estadísticas. No tiene un modelo mental del mundo, no entiende la física, no razona causalmente. Las capacidades emergentes son patrones estadísticos sofisticados, no comprensión en el sentido cognitivo.

Error 2: "Más contexto siempre es mejor." El coste de la atención crece cuadráticamente con el contexto. Además, el fenómeno lost-in-the-middle muestra que los modelos prestan menos atención a la información en el centro del contexto que al principio y al final. Un contexto grande y mal estructurado puede producir peores resultados que un contexto más corto y bien seleccionado.

Error 3: "Los parámetros del Transformer son todos iguales en importancia." No lo son. Las cabezas de atención en las primeras capas aprenden patrones sintácticos; las de las capas intermedias aprenden semántica; las de las últimas capas aprenden aspectos específicos de la tarea. Las matrices del FFN almacenan conocimiento factual. Esta estructura interna es relevante para técnicas de fine-tuning eficiente como LoRA.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Entender la arquitectura Transformer tiene aplicaciones prácticas directas en el diseño de sistemas. Primero: el coste cuadrático del contexto significa que debes gestionar activamente qué información incluir en el contexto. En sistemas RAG, recuperar solo los chunks más relevantes en lugar de pasar documentos completos no es solo una práctica de ingeniería: es una consecuencia directa del coste de la atención.

Segundo: el lost-in-the-middle significa que las instrucciones más importantes (el system prompt, los criterios de evaluación, las restricciones) deben estar al principio o al final del contexto, no enterradas en el medio. Tercero: entender que los encoders (BERT) son mejores para embeddings y clasificación, y los decoders (GPT) son mejores para generación, permite elegir el tipo de modelo correcto para cada caso de uso.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M1-T9 (Mecanismo de atención): profundiza en el componente más importante del Transformer.
- M1-T10 (Tokenización): el primer paso del procesamiento del Transformer.
- M4 (Deep Learning): el Transformer en el contexto de la evolución de las arquitecturas de deep learning.
- M5 (LLMs): los LLMs son Transformers preentrenados a gran escala con ajuste por retroalimentación humana.

SECCIÓN 8 · RESUMEN EJECUTIVO

El Transformer (Vaswani et al., 2017) es la arquitectura que subyace a todos los LLMs modernos. Se construye apilando bloques con dos componentes: multi-head self-attention (que permite a cada token atender a todos los demás) y FFN (que aplica transformaciones no lineales aprendidas). Las conexiones residuales permiten entrenar redes muy profundas. La variante decoder-only (GPT, Llama) usa masked attention para generación autoregresiva. El coste cuadrático de la atención con el contexto es la limitación técnica central que determina el diseño de sistemas de IA basados en LLMs.